1. **Process Creation in Unix Shell:** Creating Child Processes and Displaying Parent-Child Relationships Using ps.

#!/bin/bash

echo "Parent PID: $$" (current shell PID)

sleep 20 &

echo “Child PID: $!” (child PID- sleep)

ps -o pid,ppid,cmd

- `$$` → gives Parent PID
  `sleep 30 &` → creates child process
  `$!` → gives child PID (pid of last background process)
  `ps -o pid,ppid,cmd` → shows process details (parent–child clearly)

  ps --ppid $$ - all child process of the shell

```
thenmozhi@DESKTOP-3L09T9L:/mnt/c/Users/LENOVO/desktop$ nano ps1.sh
thenmozhi@DESKTOP-3L09T9L:/mnt/c/Users/LENOVO/desktop$ chmod +x ps1.sh
thenmozhi@DESKTOP-3L09T9L:/mnt/c/Users/LENOVO/desktop$ ./ps1.sh
Parent PID: 1815
Child PID:1816
thenmozhi@DESKTOP-3L09T9L:/mnt/c/Users/LENOVO/desktop$ ps -o pid,ppid,cmd
    PID    PPID CMD
   1739    1738 -bash
   1816    1738 sleep 30
   1817    1739 ps -o pid,ppid,cmd
```

```
thenmozhi@DESKTOP-3L09T9L:/mnt/c/Users/LENOVO/desktop$ sleep 30
^Z
[1]+  Stopped                 sleep 30
thenmozhi@DESKTOP-3L09T9L:/mnt/c/Users/LENOVO/desktop$ sleep 50
^Z
[2]+  Stopped                 sleep 50
thenmozhi@DESKTOP-3L09T9L:/mnt/c/Users/LENOVO/desktop$ pstree -p $$
bash(1739)───pstree(1893)
           ├─sleep(1891)
           └─sleep(1892)
thenmozhi@DESKTOP-3L09T9L:/mnt/c/Users/LENOVO/desktop$ pstree -p
systemd(1)───agetty(204)
           ├─agetty(215)
           ├─cron(179)
           ├─dbus-daemon(180)
           ├─init-systemd(Ub(2)───SessionLeader(1737)───Relay(1739)(1738)───bash(1739)───pstree(1894)
                                                                                       ├─sleep(1891)
                                                                                       └─sleep(1892)
                                 ├─init(7)───{init}(8)
                                 ├─login(328)───bash(428)
                                 └─{init-systemd(Ub}(9)
           ├─polkitd(1056)───{polkitd}(1058)
                             ├─{polkitd}(1059)
                             └─{polkitd}(1060)
```

2. **Process Control**: Create a shell script that demonstrates process control commands like bg, fg, jobs, and kill. Implement a simple background task and manage its execution.

```
thenmozhi@DESKTOP-3L09T9L:/mnt/c/Users/LENOVO/desktop$ jobs
thenmozhi@DESKTOP-3L09T9L:/mnt/c/Users/LENOVO/desktop$ sleep 500
^Z
[1]+  Stopped                 sleep 500
thenmozhi@DESKTOP-3L09T9L:/mnt/c/Users/LENOVO/desktop$ sleep 300
^Z
[2]+  Stopped                 sleep 300
thenmozhi@DESKTOP-3L09T9L:/mnt/c/Users/LENOVO/desktop$ cat
^Z
[3]+  Stopped                 cat
thenmozhi@DESKTOP-3L09T9L:/mnt/c/Users/LENOVO/desktop$ jobs
[1]   Stopped                 sleep 500
[2]-  Stopped                 sleep 300
[3]+  Stopped                 cat
thenmozhi@DESKTOP-3L09T9L:/mnt/c/Users/LENOVO/desktop$ bg %2
[2]- sleep 300 &
thenmozhi@DESKTOP-3L09T9L:/mnt/c/Users/LENOVO/desktop$ jobs
[1]-  Stopped                 sleep 500
[2]   Running                 sleep 300 &
[3]+  Stopped                 cat
thenmozhi@DESKTOP-3L09T9L:/mnt/c/Users/LENOVO/desktop$ fg %3
cat
i am running in forground
i am running in forground
^Z
[3]+  Stopped                 cat
thenmozhi@DESKTOP-3L09T9L:/mnt/c/Users/LENOVO/desktop$ jobs
[1]-  Stopped                 sleep 500
[2]   Running                 sleep 300 &
[3]+  Stopped                 cat
thenmozhi@DESKTOP-3L09T9L:/mnt/c/Users/LENOVO/desktop$ kill %1

[1]-  Stopped                 sleep 500
thenmozhi@DESKTOP-3L09T9L:/mnt/c/Users/LENOVO/desktop$ jobs
[1]-  Terminated              sleep 500
[2]   Running                 sleep 300 &
[3]+  Stopped                 cat
thenmozhi@DESKTOP-3L09T9L:/mnt/c/Users/LENOVO/desktop$
```

3. **Basic Shell Script for Job Scheduling**: Develop a shell script that takes a list of commands and executes them with a specified delay using the sleep command. Demonstrate job scheduling in the shell.

```
echo "job 1: Display Date"
date
sleep 3

echo "job 2: Display current user"
whoami
sleep 3

echo "job 3: Display Calender"
cal
sleep 2

echo " job 4: list the files"
ls
sleep 3

echo "job 5: display my system date"
date +"%T"
sleep 1

echo "job 6: print working directory"
pwd

echo "All jobs are finished"
```

- Date cmd used to show the date.
- Whoami cmd shows the current user.
- Cal shows the calendar of current month

```
echo "job 1: Display Date"
date
sleep 3

echo "job 2: Display current user"
whoami
sleep 3

echo "job 3: Display Calender"
cal
sleep 2

echo " job 4: list the files"
ls
sleep 3

echo "job 5: display my system date"
date +"%T"
sleep 1

echo "job 6: print working directory"
pwd

echo "All jobs are finished"
```

```
thenmozhi@DESKTOP-3L09T9L:/mnt/c/Users/LENOVO/desktop$ ./lab3.sh
“ job 1: Display Date”
Thu Feb 26 20:34:12 UTC 2026
“ job 2: Display current user”
thenmozhi
“ job 3: Display Calender”
./lab3.sh: line 8: cal: command not found
“ All jobs are finished”
thenmozhi@DESKTOP-3L09T9L:/mnt/c/Users/LENOVO/desktop$ sudo apt update
[sudo] password for thenmozhi:
Hit:1 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:2 http://archive.ubuntu.com/ubuntu noble InRelease
Get:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Hit:4 http://archive.ubuntu.com/ubuntu noble-backports InRelease
Get:5 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1800 kB]
Get:6 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [1558 kB]
Fetched 3484 kB in 7s (491 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
115 packages can be upgraded. Run 'apt list --upgradable' to see them.
thenmozhi@DESKTOP-3L09T9L:/mnt/c/Users/LENOVO/desktop$ sudo apt install bsdmainutils
```

```
thenmozhi@DESKTOP-3L09T9L:/mnt/c/Users/LENOVO/desktop$ ./lab3.sh
“ job 1: Display Date”
Thu Feb 26 20:46:03 UTC 2026
“ job 2: Display current user”
thenmozhi
“ job 3: Display Calender”
   February 2026
Su Mo Tu We Th Fr Sa
 1  2  3  4  5  6  7
 8  9 10 11 12 13 14
15 16 17 18 19 20 21
22 23 24 25 26 27 28


“ job 4: list the files”
'B.Tech OS.docx'                                                    'TOMORRWOW SCHEDULE.xls
'BCA 2025 - 2026 Result Analysis.docx'                              'Thesis Mail.jpg'
'BCA Faculty Subject Allotment,Workload & Additional work_AY_2025-26_Even.xlsx'   abc.sh
```

4. **File Operations**: Create a shell script that performs file operations, such as creating, copying, moving, and deleting files using commands like touch, cp, mv, and rm. Include error handling to manage file operations gracefully.

In shell scripting (file operations), -f and **-e** are file test operators used with if condition.

-f → Checks if file exists AND is a regular file - if [-f filename] □ normal file

-e → Checks if file exists (any type) - if [-e filename] □ File exists (file / directory / link anything)

```
#!/bin/bash

while true
do
echo " -------------- FILE OPERATIONS ----------"
echo "1. Create File"
echo "2. Copy File"
echo "3. Move File"
echo "4. Delete File"
echo "5. Display File"
echo "6. Exit"
echo -n "Enter your choice: "
read choice

if [ $choice -eq 1 ]; then
read -p "Enter filename to create: " fname          # fname - variable

if [ -f "$fname" ]; then                    #$fname – value stored inside the variable
echo "Error: File already exists!"
else
read -p "Enter content: " content
echo "$content" > "$fname"
echo "File '$fname' created successfully."
fi

elif [ $choice -eq 2 ]; then
read -p "Enter source filename: " source
read -p "Enter destination filename: " dest

if [ -f "$source" ]; then
cp "$source" "$dest"
echo "File copied successfully."
else
echo "Error: Source file does not exist!"
fi

elif [ $choice -eq 3 ]; then
read -p "Enter filename to move: " source
read -p "Enter new filename: " dest

if [ -f "$source" ]; then
mv "$source" "$dest"
```

```
echo "File moved successfully."
else
echo "Error: File does not exist!"
fi

elif [ $choice -eq 4 ]; then
read -p "Enter filename to delete: " fname

if [ -f "$fname" ]; then
rm "$fname"
echo "File deleted successfully."
else
echo "Error: File does not exist!"
fi

elif [ $choice -eq 5 ]; then
read -p "Enter filename to display: " fname

if [ -f "$fname" ]; then
echo "Content of '$fname':"
cat "$fname"
else
echo "Error: File does not exist!"
fi

elif [ $choice -eq 6 ]; then
echo "Exiting program..."
break

else
echo "Invalid choice!"
fi

echo ""
done
```

```
thenmozhi@DESKTOP-3L09T9L:/mnt/c/Users/
====== FILE OPERATIONS ======
1. Create File
2. Copy File
3. Move File
4. Delete File
5. Display File
6. Exit
Enter your choice: 1
Enter filename to create: Mithran
Enter content: I am newly created file
File 'Mithran' created successfully.

====== FILE OPERATIONS ======
1. Create File
2. Copy File
3. Move File
4. Delete File
5. Display File
6. Exit
Enter your choice: 5
Enter filename to display: mithran
Content of 'mithran':
I am newly created file

====== FILE OPERATIONS ======
1. Create File
2. Copy File
3. Move File
4. Delete File
5. Display File
6. Exit
Enter your choice: 2
Enter source filename: mithran
```

```
Content of 'mithran':
I am newly created file

====== FILE OPERATIONS ======
1. Create File
2. Copy File
3. Move File
4. Delete File
5. Display File
6. Exit
Enter your choice: 2
Enter source filename: mithran
Enter destination filename: pramu
File copied successfully.

====== FILE OPERATIONS ======
1. Create File
2. Copy File
3. Move File
4. Delete File
5. Display File
6. Exit
Enter your choice: 5
Enter filename to display: pramu
Content of 'pramu':
I am newly created file

====== FILE OPERATIONS ======
1. Create File
2. Copy File
3. Move File
4. Delete File
5. Display File
6. Exit
Enter your choice: 3
Enter filename to move: pramu
Enter new filename: mehann
File moved successfully.
```

```
File deleted successfully.

====== FILE OPERATIONS ======
1. Create File
2. Copy File
3. Move File
4. Delete File
5. Display File
6. Exit
Enter your choice: 5
Enter filename to display: mehann
Error: File does not exist!

====== FILE OPERATIONS ======
1. Create File
2. Copy File
3. Move File
4. Delete File
5. Display File
6. Exit
Enter your choice: 6
Exiting program...
thenmozhi@DESKTOP-3L09T9L:/mnt/c/Users/LENOVO/desktop$ |
```

5. File Permissions: Write a shell script to modify file permissions and ownership using chmod and chown. Include user input for the filename and desired permissions and display the new file permissions.

Chmod – Change file permissions

```
#!/bin/bash

echo "---- File Permission & Ownership ----"

# Get filename
echo "Enter file name:"
read file

# Check file exists
if [ ! -f "$file" ]; then
```

```
    echo "File not found!"
    exit
fi

# Get permission
echo "Enter permission (e.g., 755):"
read perm

# Change permission
chmod $perm $file

# Get owner
echo "Enter new owner name:"
read owner

# Change ownership
chown $owner $file

# Display new permissions
echo "Updated file details:"
ls -l $file
```

Each number = **Owner | Group | Others**

Meaning of file permissions

Number	**Meaning**
7	Read + Write + Execute (rwx)
6	Read + Write (rw-)

Number	Meaning
5	Read + Execute (r-x)
4	Read only (r--)
0	No permission (---)

777 → Everyone can read, write, execute

755 → Owner full, others read + execute

644 → Owner read/write, others read only

6. Directory Management: Develop a shell script that creates, lists, and deletes directories. Use commands like mkdir, ls, and rmdir, and include user prompts for directory names.

```bash
#!/bin/bash

echo "Lab 6: Directory Management"

while true
do
    echo ""
    echo "1. Create Directory"
    echo "2. List Directories"
    echo "3. Delete Directory"
    echo "4. Exit"

    echo "Enter your choice:"
    read choice

    case $choice in

    1)
        echo "Enter directory name to create:"
        read dirname
        mkdir "$dirname"
        if [ $? -eq 0 ]; then
            echo "Directory '$dirname' created successfully."
        else
            echo "Error creating directory."
        fi
        ;;

    2)
        echo "Directories in current location:"
        ls -d */
        ;;
```

```
    3)
        echo "Enter directory name to delete:"
        read dirname
        rmdir "$dirname"
        if [ $? -eq 0 ]; then
            echo "Directory '$dirname' deleted successfully."
        else
            echo "Error deleting directory (may not be empty)."
        fi
        ;;

    4)
        echo "Exiting program..."
        break
        ;;

    *)
        echo "Invalid choice. Try again!"
        ;;

    esac
done
```

Output :

```
henmozhi@DESKTOP-3L09T9L:/mnt/c/Users/LENOVO/desktop$ nano lab6.sh
henmozhi@DESKTOP-3L09T9L:/mnt/c/Users/LENOVO/desktop$ chmod +x lab6.sh
henmozhi@DESKTOP-3L09T9L:/mnt/c/Users/LENOVO/desktop$ ./lab6.sh
ab 6: Directory Management

. Create Directory
. List Directories
. Delete Directory
. Exit
nter your choice:

nter directory name to create:
henmozhi
irectory 'thenmozhi' created successfully.

. Create Directory
. List Directories
. Delete Directory
. Exit
nter your choice:

irectories in current location:
AI Workshop/'   JNB/         'Ph.D Thesis Evaluation/'  'RV Conference/'  'Syllabus Framing/'   thenmozhi/
EVEN/           Manuscript/   Print/                     RV/               'apple till 24/'

. Create Directory
. List Directories
. Delete Directory
. Exit
nter your choice:

nter directory name to delete:
henmozhi
irectory 'thenmozhi' deleted successfully.

. Create Directory
. List Directories
. Delete Directory
```

Practice:

```
1)      # Create a new directory
        create_directory
        ;;
2)      # List existing directories
        list_directories
        ;;
3)      # Delete an empty directory
        delete_directory
        ;;
4)      # Delete a non-empty directory
        delete_non_empty_directory
        ;;
5)      # Exit the script
        echo "Exiting the script. Goodbye!"
        exit 0
        ;;
*)      # Invalid option
        echo "Invalid option. Please select
        ;;
```

7. Dynamic Memory Check: Write a shell script that monitors memory usage on the system using commands like free and top. Display the current memory usage and available memory.

Memory usage → shows how much RAM is being used by the system

Free memory → available RAM for new processes

free command → gives summary of total, used, and free memory

top command → shows real-time system usage (CPU + memory)

-h option in free → displays values in **human-readable format (MB/GB)**

free -h → shows memory in GB/MB

top -b -n 1 | grep "MiB Mem"

 # [Memory measured in Mebibytes (MiB) , 1 MiB = 1024 × 1024 bytes]

- top → shows system performance
- -b → batch mode (no interactive screen)
- -n 1 → runs only **once**
- grep → filters only **memory line**

awk → extracts specific column (available memory)

```
#!/bin/bash

echo "Lab 7: Memory Usage Monitor"

echo ""
echo "Current Memory Status (using free command):"
free -h

echo ""
echo "Detailed Memory Info (using top command):"
top -b -n 1 | grep "MiB Mem"

echo ""
echo "Available Memory:"
free -h | grep "Mem" | awk '{print "Available: " $7}'
```

Output:

```
thenmozhi@DESKTOP-3L09T9L:/mnt/c/Users/LENOVO/desktop$ nano lab7.sh
thenmozhi@DESKTOP-3L09T9L:/mnt/c/Users/LENOVO/desktop$ chmod +x lab7.sh
thenmozhi@DESKTOP-3L09T9L:/mnt/c/Users/LENOVO/desktop$ ./lab7.sh
Lab 7: Memory Usage Monitor

Current Memory Status (using free command):
               total        used        free      shared  buff/cache   available
Mem:           3.7Gi       468Mi       3.2Gi       3.4Mi       187Mi       3.3Gi
Swap:          1.0Gi          0B       1.0Gi

Detailed Memory Info (using top command):
MiB Mem :   3833.2 total,   3318.4 free,    468.2 used,    188.2 buff/cache

Available Memory:
Available: 3.3Gi
```

8. Environment Variables: Create a shell script that sets, exports, and displays environment variables. Use printenv and env to show the current environment settings.

Environment variables are system variables that store important information

Used by OS and programs (e.g., PATH, HOME, USER)

set → creates a variable (local)

export → makes variable available to **child processes**

[makes it global (environment variable)]

printenv → displays environment variables (shows specific variable)

env → shows all environment settings (displays all environment variables)

Feature	Local Variable	Environment Variable
Scope	Current shell only	Shell + child processes
Command	var=value	export var=value
Visibility	Limited	Global (within session)

```bash
#!/bin/bash

echo "Lab 8: Environment Variables Demo "

# Setting variable
name="Thenmozhi"
course="Operating Systems"

echo ""
echo "Local Variables:"
echo "Name: $name"
echo "Course: $course"

# Exporting variables
export name
export course

echo ""
echo "Using printenv command:"
printenv name
printenv course

echo ""
echo "All Environment Variables (using env):"
env
```

Output:

```
thenmozhi@DESKTOP-3L09T9L:/mnt/c/Users/LENOVO/desktop$ nano lab8.sh
thenmozhi@DESKTOP-3L09T9L:/mnt/c/Users/LENOVO/desktop$ chmod +x lab8.sh
thenmozhi@DESKTOP-3L09T9L:/mnt/c/Users/LENOVO/desktop$ ./lab8.sh
===== Environment Variables Demo =====

Local Variables:
Name: Thenmozhi
Course: Operating Systems

Using printenv command:
Thenmozhi
Operating Systems

All Environment Variables (using env):
SHELL=/bin/bash
WSL2_GUI_APPS_ENABLED=1
WSL_DISTRO_NAME=Ubuntu
NAME=DESKTOP-3L09T9L
PWD=/mnt/c/Users/LENOVO/desktop
LOGNAME=thenmozhi
MOTD_SHOWN=update-motd
HOME=/home/thenmozhi
LANG=C.UTF-8
WSL_INTEROP=/run/WSL/318_interop
LS_COLORS=rs=0:di=01;34:ln=01;36:mh=00:pi=40;33:so=01;35:do=01;35:bd=40;33;01:cd=40;33;01:or=40;31;01:m
44:ex=01;32:*.tar=01;31:*.tgz=01;31:*.arc=01;31:*.arj=01;31:*.taz=01;31:*.lha=01;31:*.lz4=01;31:*.lzh=0
:*.t7z=01;31:*.zip=01;31:*.z=01;31:*.dz=01;31:*.gz=01;31:*.lrz=01;31:*.lz=01;31:*.lzo=01;31:*.xz=01;31:
=01;31:*.tbz2=01;31:*.tz=01;31:*.deb=01;31:*.rpm=01;31:*.jar=01;31:*.war=01;31:*.ear=01;31:*.sar=01;31:
o=01;31:*.7z=01;31:*.rz=01;31:*.cab=01;31:*.wim=01;31:*.swm=01;31:*.dwm=01;31:*.esd=01;31:*.avif=01;35:
.gif=01;35:*.bmp=01;35:*.pbm=01;35:*.pgm=01;35:*.ppm=01;35:*.tga=01;35:*.xbm=01;35:*.xpm=01;35:*.tif=01
:*.mng=01;35:*.pcx=01;35:*.mov=01;35:*.mpg=01;35:*.mpeg=01;35:*.m2v=01;35:*.mkv=01;35:*.webm=01;35:*.we
1;35:*.vob=01;35:*.qt=01;35:*.nuv=01;35:*.wmv=01;35:*.asf=01;35:*.rm=01;35:*.rmvb=01;35:*.flc=01;35:*.a
5:*.xcf=01;35:*.xwd=01;35:*.yuv=01;35:*.cgm=01;35:*.emf=01;35:*.ogv=01;35:*.ogx=01;35:*.aac=00;36:*.au=
36:*.mka=00;36:*.mp3=00;36:*.mpc=00;36:*.ogg=00;36:*.ra=00;36:*.wav=00;36:*.oga=00;36:*.opus=00;36:*.sp
crdownload=00;90:*.dpkg-dist=00;90:*.dpkg-new=00;90:*.dpkg-old=00;90:*.dpkg-tmp=00;90:*.old=00;90:*.ori
orig=00;90:*.rpmsave=00;90:*.swp=00;90:*.tmp=00;90:*.ucf-dist=00;90:*.ucf-new=00;90:*.ucf-old=00;90:
WAYLAND_DISPLAY=wayland-0
LESSCLOSE=/usr/bin/lesspipe %s %s
TERM=xterm-256color
LESSOPEN=| /usr/bin/lesspipe %s
```

9. I/O Redirection in Unix Shell: Manipulating Data Streams with Enhanced File Operations.

I/O Redirection is used to control input and output of commands

Default:

- Input → Keyboard
- Output → Screen

Redirection allows:

- Store output in files
- Take input from files

> → writes to file

>> → adds to file

< → reads from file

2> → saves error into file

Number	Name	Meaning
0	stdin	Input (keyboard)
1	stdout	Normal output
2	stderr	Error output

```bash
#!/bin/bash

echo "Lap 9: I/O Redirection Demo "

# Output redirection (overwrite)
echo "Hello, this is OS Lab" > file1.txt
echo "Content written to file1.txt"

# Append output
echo "Appending new line" >> file1.txt
echo "Content appended to file1.txt"

# Input redirection
echo ""
echo "Reading from file1.txt using input redirection:"
cat < file1.txt

# Error redirection
echo ""
echo "Trying to read non-existing file:"
cat file_not_exist.txt 2> error.txt
echo "Error message stored in error.txt"

# Display error file
echo ""
echo "Error file content:"
cat error.txt
```

Output:

```
thenmozhi@DESKTOP-3L09T9L:/mnt/c/Users/LENOVO/desktop$ nano lab9.sh
thenmozhi@DESKTOP-3L09T9L:/mnt/c/Users/LENOVO/desktop$ ./lab9.sh
Lap 9: I/O Redirection Demo
Content written to file1.txt
Content appended to file1.txt

Reading from file1.txt using input redirection:
Hello, this is OS Lab
Appending new line

Trying to read non-existing file:
Error message stored in error.txt

Error file content:
cat: file_not_exist.txt: No such file or directory
thenmozhi@DESKTOP-3L09T9L:/mnt/c/Users/LENOVO/desktop$
```

10. Command Piping in Unix Shell: Exploring Operating System Features.

```
#!/bin/bash
# Filename: pipe_demo.sh
# Purpose: Demonstrate command piping in Unix shell

echo "Command Piping Demo "

# 1. Create a sample file
echo -e "apple\nbanana\napple\norange\nbanana\napple" > fruits.txt
echo "Sample file 'fruits.txt' created with content:"
cat fruits.txt
echo

# 2. Count total lines using piping
total_lines=$(cat fruits.txt | wc -l)
echo "Total lines in fruits.txt: $total_lines"

# 3. Count occurrences of 'apple'
apple_count=$(cat fruits.txt | grep "apple" | wc -l)
echo "Number of times 'apple' appears: $apple_count"

# 4. Sort the file and remove duplicates
echo "Sorted unique fruits:"
cat fruits.txt | sort | uniq
echo
```

```
# 5. Search for 'banana' in file
echo "Lines containing 'banana':"
cat fruits.txt | grep "banana"

# Cleanup
rm fruits.txt
echo
echo "Demo complete. Sample file removed."
```

Output

```
thenmozhi@DESKTOP-3L09T9L:/mnt/c/Users/LENOVO/desktop$ nano lab10.txt
thenmozhi@DESKTOP-3L09T9L:/mnt/c/Users/LENOVO/desktop$ chmod +x lab10.txt
thenmozhi@DESKTOP-3L09T9L:/mnt/c/Users/LENOVO/desktop$ ./lab10.txt
Command Piping Demo
Sample file 'fruits.txt' created with content:
apple
banana
apple
orange
banana
apple

Total lines in fruits.txt: 6
Number of times 'apple' appears: 3
Sorted unique fruits:
apple
banana
orange

Lines containing 'banana':
banana
banana

Demo complete. Sample file removed.
```

Thank you..